Callbacks can be passive, monitoring activity performed by a controller. They can also take a more active part in request handling. If a before action callback returns `false`, then processing of the callback chain terminates and the action isn't run. A callback may also render output or redirect requests, in which case the original action never gets invoked.

Callback declarations also accept blocks and the names of classes. If a block is specified, it'll be called with the current controller as a parameter. If a class is given, its `filter` class method will be called with the controller as a parameter.

By default, callbacks apply to all actions in a controller (and any subclasses of that controller). You can modify this with the `:only` option, which takes one or more actions on which the callback is invoked, and the `:except` option, which lists actions to be excluded from callback.

The `before_action` and `after_action` declarations append to the controller's chain of callbacks. Use the variants `prepend_before_action` and `prepend_after_action` to put callbacks at the front of the chain.

After callbacks can be used to modify the outbound response, changing the headers and content if required. Some applications use this technique to perform global replacements in the content generated by the controller's templates (for example, by substituting a customer's name for the string `<customer/>` in the response body). Another use might be compressing the response if the user's browser supports it.

Around callbacks wrap the execution of actions. You can write an around callback in two different styles. In the first, the callback is a single chunk of code. That code is called before the action is executed. If the callback code invokes `yield`, the action is executed. When the action completes, the callback code continues executing.